Eduvos
Your Education. Your Future.

Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Mowbray

Faculty: Information Technology

Module Code: ITEPA3-33

Group: Individual

Lecturer's Name:

Student Full Name: Joshua Nehohwa

Student Number: EDUV4 948 467

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date

Lecturer's Comments:

Marks Awarded: %

Signature	Date

ITEPA3-33 Practical Assignment: EduCore Enterprise Application

Python enterprise application prototype, evidence and design evaluation

WORKING DRAFT - AI ASSISTANCE IS DISCLOSED IN THE REPORT AND AI USE LOG

Executive summary

EduCore is a Python enterprise-application prototype for a growing South African training provider. It centralises learners, courses, registrations, assessments and support tickets while demonstrating maintainable object-oriented design, three enterprise design patterns, thread-safe concurrent processing, structured monitoring, automated testing, profiling and a migration path to microservices.

The implementation is deliberately modular. Domain entities contain validation and business meaning; services coordinate workflows; strategies and factories isolate variable behaviour; and Bugzot captures operational evidence.
The final test run completed 32 tests successfully with 92% measured coverage. A contention test confirmed that concurrent registration never exceeded course capacity and did not create duplicate records.

Contents

System boundaries

This submission is a working command-line prototype supported by interface mockups. Persistence remains in memory so that the assessed enterprise mechanisms are visible and testable. The report distinguishes prototype behaviour from future production requirements such as an external database, authentication, deployment and distributed infrastructure.

1. Deliverable 1: Foundational components

1.1 Domain model implementation

The domain model uses validated dataclasses with explicit state enums. Each class has one central responsibility and exposes operations that preserve its invariants. Composition and identifier relationships keep entities understandable without tightly coupling every object to every other object.

Class	Responsibility and main validation	Relationship
Learner	Stores identity, normalises email and rejects missing or invalid values.	A learner participates in registrations and owns support tickets.
Course	Stores title and a strictly positive integer capacity.	Tracks assessment identifiers and receives registrations.
Registration	Immutable record containing learner, course, status and UTC creation time.	Joins one learner to one course.
Assessment	Validates title, maximum mark and score boundaries.	Belongs to a course and delegates result calculation to a strategy.
SupportTicket	Controls category, priority and open-to-resolved workflow.	Belongs to a learner and is created through the factory.

Object-oriented principles

-Encapsulation: validation is performed inside domain constructors and state-changing methods.

-Abstraction: assessment calculations depend on the AssessmentStrategy interface rather than a specific algorithm.

-Polymorphism: each strategy provides the same calculate interface but returns its algorithm-specific result.-Single

responsibility: registration workflow, monitoring and entity state are held in separate modules.

```
course.add_assessment(assessment.assessment_id)
registration = Registration(learner.learner_id, course.course_id)
ticket.start_work()
ticket.resolve()
```

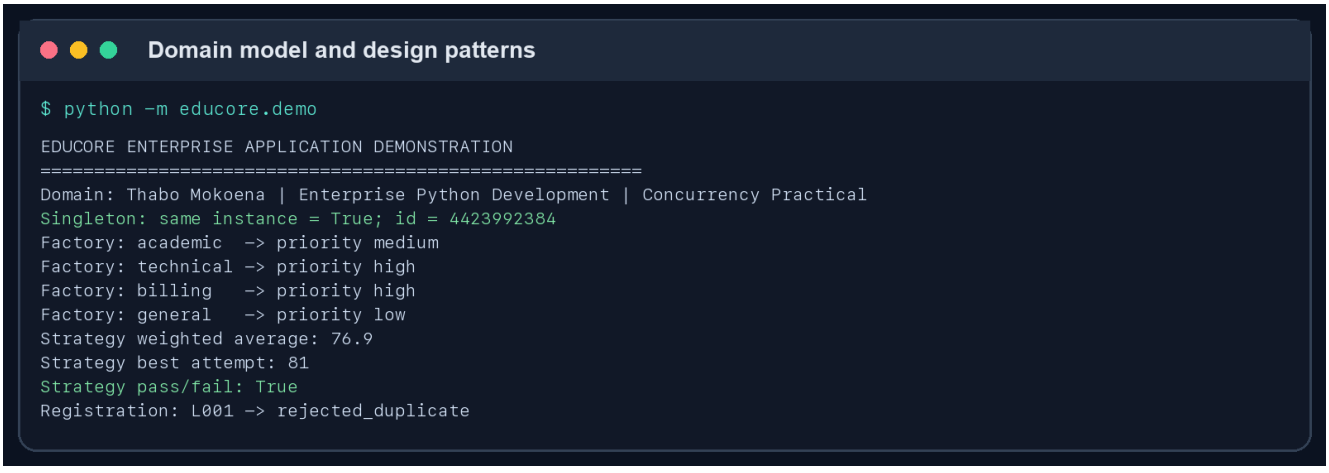

Figure 1. Successful creation of related domain objects and execution of all required patterns.

1.2 Design pattern implementation

Pattern	Implementation	Enterprise relevance
Singleton	ApplicationConfig uses a class-level instance and double-checked lock.	All services observe one configuration without repeated file reads or inconsistent settings.

Pattern	Implementation	Enterprise relevance
Factory	SupportTicketFactory maps ticket categories to controlled priority defaults.	New ticket types can be added without scattering construction rules through controllers.
Strategy	WeightedAverage, BestAttempt and PassFail implement AssessmentStrategy.	Assessment policy can vary by programme without changing the Assessment entity or calling workflow.

The Singleton lock protects first-time creation if multiple threads request configuration simultaneously. The Factory rejects unsupported categories, while the Strategy implementations validate score and weight boundaries. The demonstration proves that both configuration requests return the same object identity and that each algorithm produces a distinct, correct result.

2. Deliverable 2: Scalable registration

2.1 Registration processing engine

RegistrationService owns learner and course indexes, confirmed registration records and per-course counters. A request receives a correlation ID and one explicit outcome: confirmed, rejected_duplicate, rejected_capacity or rejected_invalid. The service therefore supports both accurate records and a complete processing summary.

Processing stage	Rule	Failure outcome
Identity validation	Learner and course must exist; learner must be active.	rejected_invalid
Uniqueness	The learner-course key must not already exist.	rejected_duplicate
Capacity	Confirmed count must remain below course capacity.	rejected_capacity
Commit	Insert the immutable registration and increment the course counter.	confirmed

The demonstration processes 16 requests, including a deliberate duplicate, against a course with capacity 10. Because request completion order is deliberately nondeterministic, the duplicate request may win the race and the earlier request may be reported as duplicate; the invariant remains that only one L001-PY701 record exists.

2.2 Scalable data modelling

Approach	Lookup	Advantages	Limitations
List of registrations	O(n)	Simple iteration and insertion.	Every duplicate check scans progressively more records.
Dictionary/set index	Average O(1)	Fast uniqueness checks and explicit composite keys.	In-memory state is process-local and non-durable.
Production database	Indexed lookup	Durability, transactions, unique constraints and multi-instance access.	Requires schema, deployment and operational management.

The prototype selects dictionaries and sets because they improve processing cost without obscuring the enterprise concepts being assessed. The same composite key maps naturally to a database unique constraint in a production migration.

2.3 Concurrent request processing

ThreadPoolExecutor represents simultaneous registration traffic. An Event releases queued work together, increasing contention. The critical section is protected by one Lock around duplicate check, capacity check and insertion. Treating this sequence atomically prevents both check-then-act races and lost capacity updates.

```
with self._lock:
if key in self._registrations:
return duplicate_result
if self._course_counts[course_id] >= course.capacity:
return capacity_result
self._registrations[key] = registration
self._course_counts[course_id] += 1
```

Risk	Failure without protection	Control
Duplicate race	Two threads both see no existing key and insert twice.	Composite-key lookup and lock around check plus insert.
Capacity race	Several threads see the last available place.	Capacity check and counter increment share the same lock.
Monitoring race	Events are lost or list state is inconsistent.	Bugzot protects its event collection with a separate lock.
Deadlock	Workers wait indefinitely.	Short non-nested critical sections; start coordination uses Event rather than a barrier.

Figure 2. Sixteen concurrent requests produce ten confirmations, five capacity rejections and one duplicate rejection; the final count equals capacity.

3. Deliverable 3: Bugzot monitoring

3.1 Monitoring subsystem

Bugzot records immutable structured events. Each event contains UTC timestamp, severity, component, event type, outcome, message, correlation ID, learner/course context and optional duration. This provides enough information to trace a request, group operational failures and reproduce the affected business context.

Event type	Trigger	Diagnostic value
validation_failure	Unknown or inactive learner, or unknown course.	Identifies invalid input and affected identifiers.
duplicate_registration	Existing learner-course composite key.	Shows attempted duplicate and protects record accuracy.
course_capacity_violation	Confirmed count equals course capacity.	Supports demand and capacity planning.
registration_success	Registration committed successfully.	Supports transaction volume and success-rate monitoring.

Events can be exported to CSV for investigation and to JSON for management reporting. Bugzot itself is independent of console presentation, which keeps monitoring reusable.

3.2 Application performance monitoring

The captured run contains 16 events: 10 confirmed and 6 rejected transactions. The success rate is 62.5%. Average measured processing time is 0.2301 ms, p95 is 0.6481 ms, and observed throughput is 12 996.51 transactions per second on this local in-memory workload.

> These figures describe a small simulator run, not production capacity. They are useful as a baseline and for regression comparison; real deployment testing would use sustained load, external storage and percentile aggregation over longer windows.

Bugzot performance report

```
$ python -m educore.demo --report-directory evidence/reports
{
  "average_duration_ms": 0.1341,
  "events_by_type": {
    "course_capacity_violation": 5,
    "duplicate_registration": 1,
    "registration_success": 10
  },
  "failed_transactions": 6,
  "maximum_duration_ms": 0.3926,
  "minimum_duration_ms": 0.0062,
  "outcomes": {
    "confirmed": 10,
    "rejected_capacity": 5,
    "rejected_duplicate": 1
  },
  "p95_duration_ms": 0.3926,
  "success_rate_percent": 62.5,
  "successful_transactions": 10,
  "throughput_transactions_per_second": 6073.03,
  "total_events": 16
}
```

Figure 3. Bugzot performance report generated from the registration run.

3.3 Performance improvement

The original duplicate-detection candidate scanned a registration list. The root cause is linear search: as the collection grows, each worst-case membership check compares against every existing registration. The implementation replaces this with a set/dictionary composite-key index, whose hash lookup is constant time on average.

Metric	Before: list	After: set	Result
Dataset size	10 000	10 000	Identical inputs
Repeated lookups	2000	2000	Identical workload
Measured seconds	0.249514	5.3e-05	99.98% faster
Functional result	Target found	Target found	Equivalent behaviour

Repeated runs will vary with hardware and background activity, but the algorithmic advantage remains. The next likely bottleneck in production is database or network latency rather than in-memory lookup.

Figure 4. Reproducible before-and-after lookup benchmark.

4. Deliverable 4: Interface design and evaluation

4.1 User interface design

The proposed desktop-first interface uses persistent role-aware navigation, consistent cards, plain labels and visible system feedback. Learners receive focused support and registration interactions; administrators receive course management and operational reporting. Required fields use labels and inline errors rather than relying on colour alone.

Requirement	Screen and interaction
Learner registration	Validated form with identity, email and course selection plus confirmation guidance.
Course management	Capacity and enrolment table, availability status and controlled edit action.
Support ticket creation	Category, automatically assigned priority, subject, description and response expectation.
Report viewing	Date filter, registration outcome visualisation, system-health summary and export action.

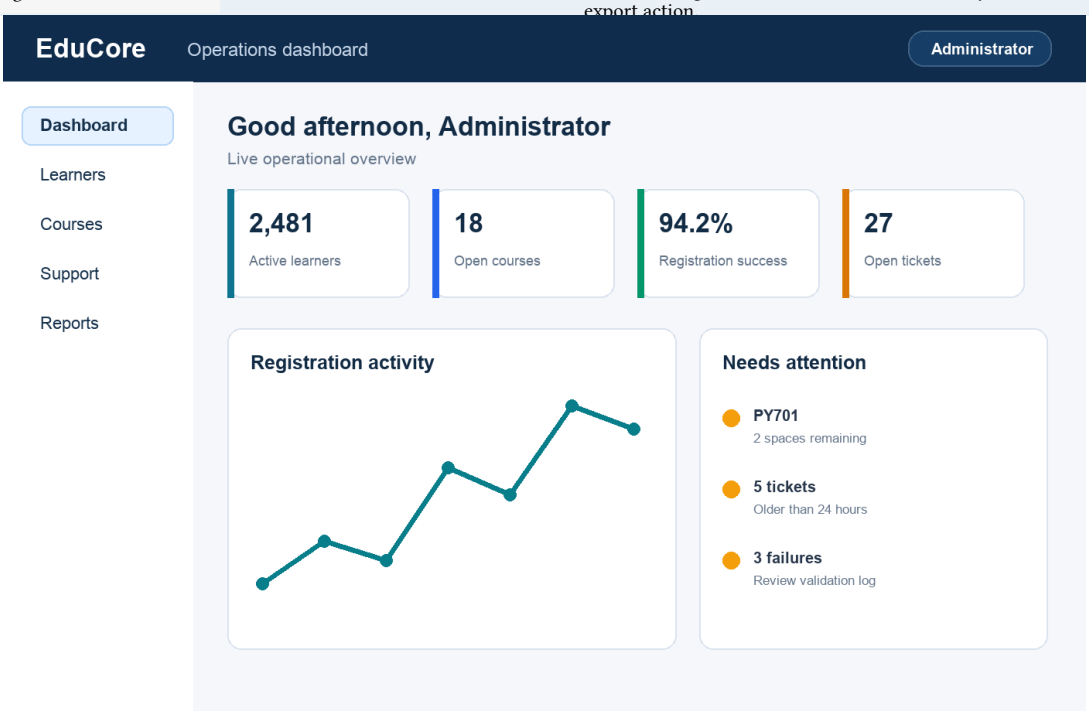

Figure 5. Administrator dashboard and primary navigation.

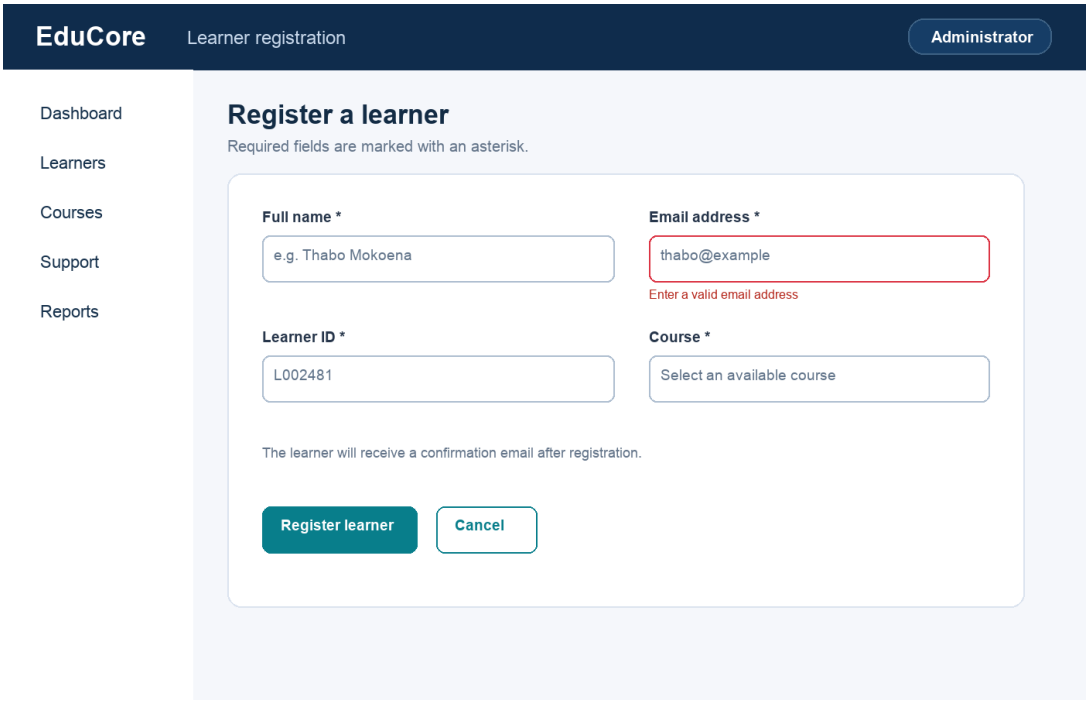

Figure 6. Learner-registration form with inline validation.

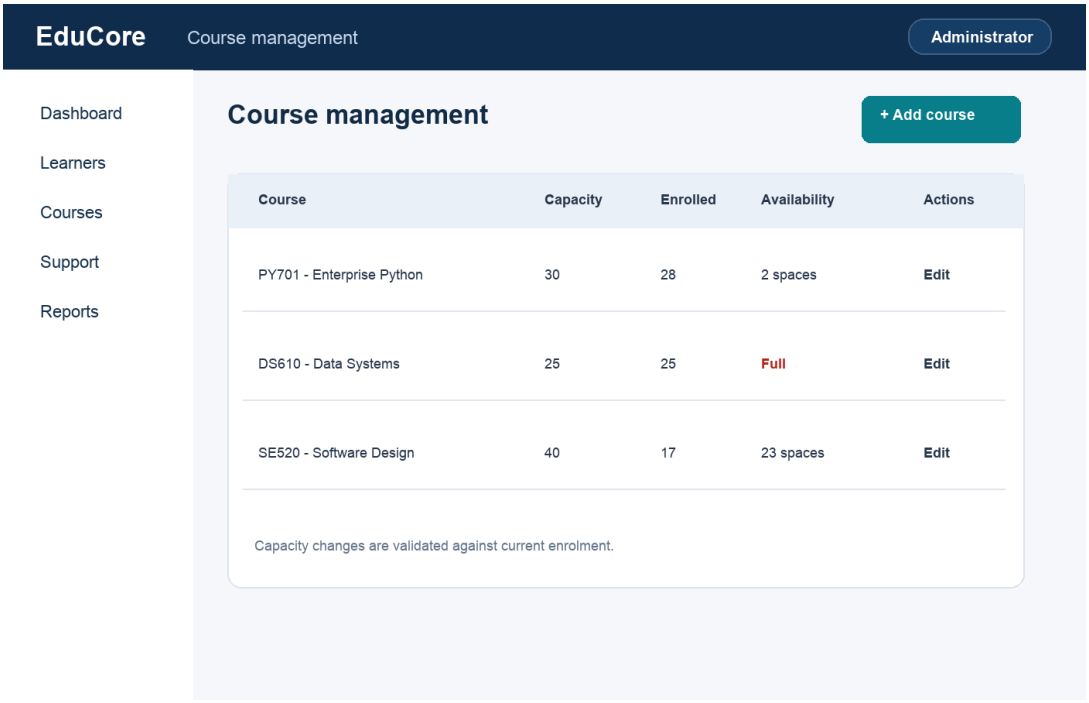

Figure 7. Course capacity and availability management.

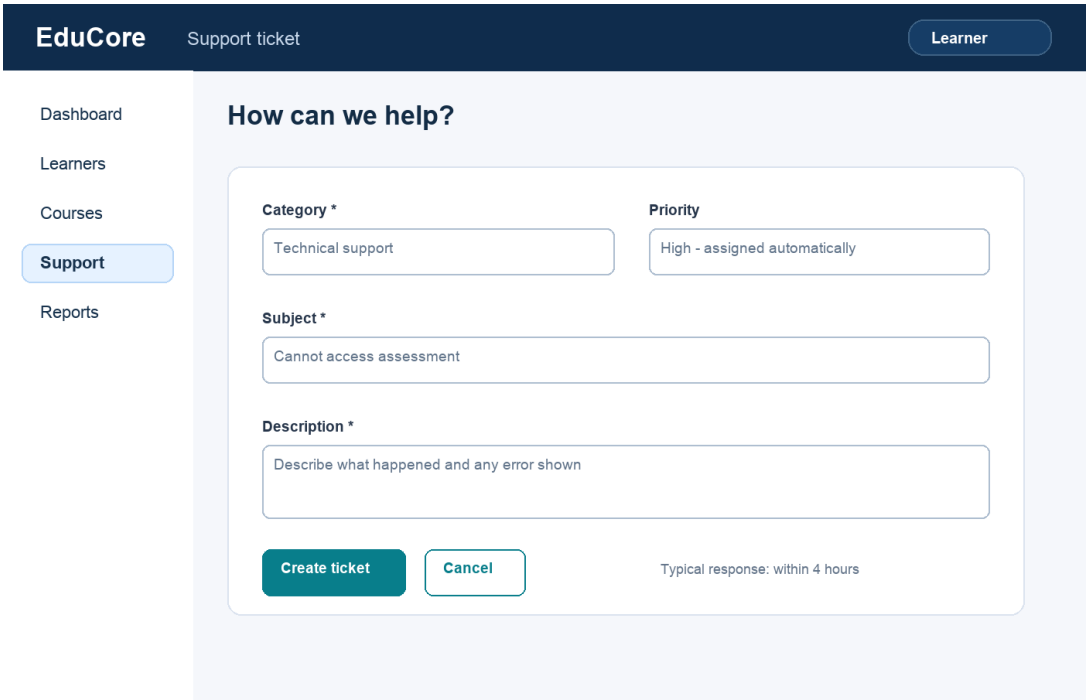

Figure 8. Learner support-ticket creation workflow.

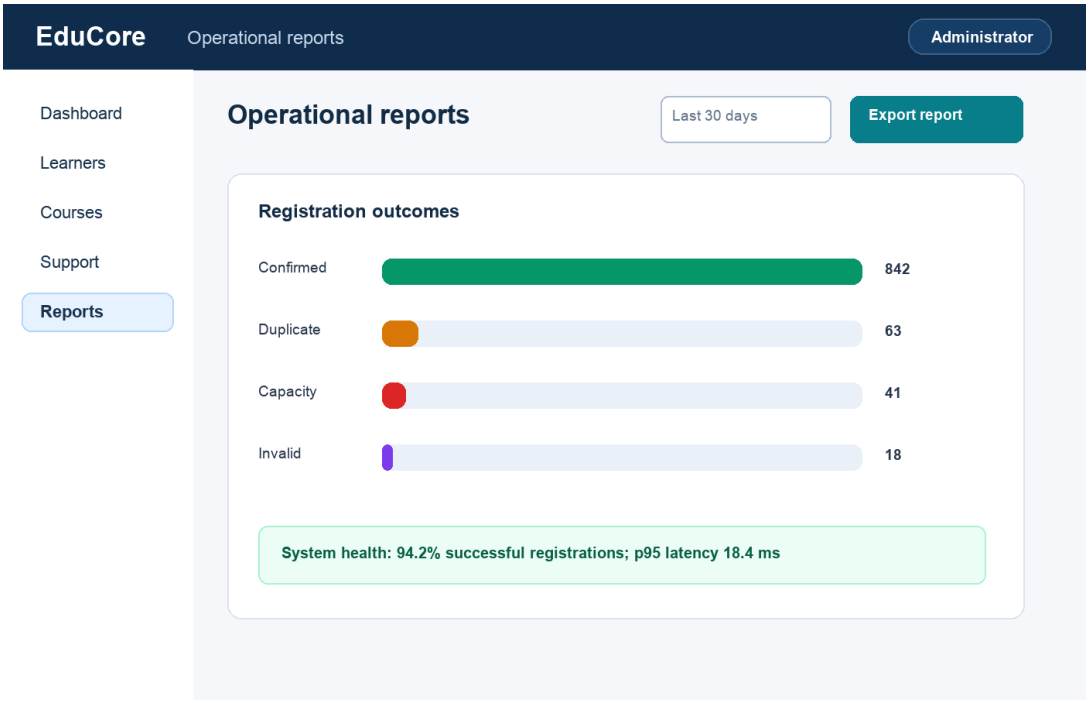

Figure 9. Operational report with outcome and health indicators.

4.2 Design decisions and evaluation

Layout and navigation: a persistent left navigation column provides predictable access while the top bar confirms the current role and context. Primary actions use consistent teal buttons and are positioned after the required information.

Validation and usability: errors appear beside the relevant field with specific correction guidance. Capacity is visible before enrolment, priority is explained, report state is summarised in text, and destructive actions are not placed beside primary actions.

Strengths	Limitation	Recommended improvement
Consistent visual hierarchy, task-focused forms, visible validation, role context and operational feedback.	The prototype is desktop-first and has not been tested with representative users or assistive technology.	Create responsive mobile variants and perform moderated usability plus WCAG keyboard/screen-reader testing before implementation.

5. Deliverable 5: Quality and future growth

5.1 Automated testing

Pytest was selected because it provides concise tests, parameterisation, clear failure output and a mature coverage ecosystem. The 32-test suite covers normal behaviour, boundaries, invalid inputs, every required pattern, business rules, concurrent invariants, structured monitoring and the benchmark harness.

Test category	Representative scenarios
Domain validation	Invalid email, empty text, zero/negative capacity, score boundaries and ticket state transition.
Patterns	Singleton identity, four Factory categories, invalid category, three strategies and invalid scores.
Registration	Success, invalid input, duplicate, capacity, summary completeness and concurrent contention.
Monitoring	Context capture, performance aggregation and JSON export.
Performance	List/set benchmark executes equivalent successful lookups.

The test suite completed with 32 passes and 92% coverage of assessed core modules. The demonstration orchestration is intentionally omitted from coverage because its behaviour is exercised through the underlying tested services and captured end-to-end output.

Automated test and coverage evidence

```
$ python -m pytest --cov=educore --cov-report=term-missing

============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
collected 32 items
tests/test_benchmark.py .                                                [  3%]
tests/test_bugzot.py ...                                                 [ 12%]
tests/test_domain.py ............                                        [ 50%]
tests/test_patterns.py ..........                                        [ 81%]
tests/test_registration.py ......                                        [100%]
================================ tests coverage ================================
src/educore/__init__.py           4      0   100%
src/educore/benchmark.py         26      1    96%   44
src/educore/bugzot.py            75     16    79%   32-34, 70-72, 108, 130-138
src/educore/domain.py           100      3    97%   49, 106, 111
src/educore/patterns.py          59      4    93%   72, 84, 86, 103
src/educore/registration.py      79      2    97%   53, 128
TOTAL                           343     26    92%
============================== 32 passed in 0.12s ==============================
```

Figure 10. Successful automated test and coverage run.

5.2 Application profiling

The application was profiled with Python cProfile using the full demonstration workload. Results were sorted by cumulative time so that work initiated through helper calls remains visible.

Benchmark list membership and demonstration/report setup dominate cumulative time; registration critical sections remain small.

The profile supports the implemented set-based optimisation. Further optimisation should focus on realistic persistence and report serialisation only after measuring a database-backed workload. Micro-optimising validated entity construction would add complexity without material benefit.

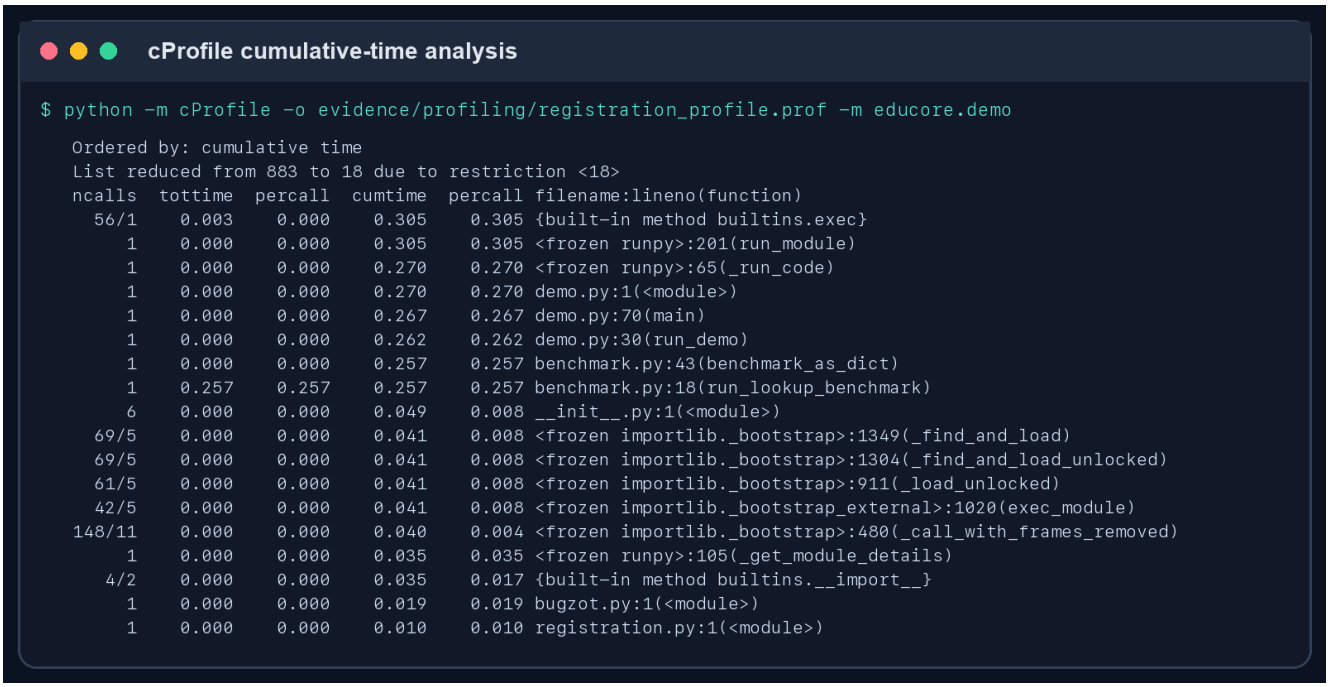

Figure 11. cProfile results sorted by cumulative time.

5.3 Microservices readiness assessment

The modular monolith provides clear candidate boundaries without prematurely introducing distributed-system cost. Each service should own its data and publish stable contracts. The Registration Service is the consistency boundary for enrolment; Reporting consumes events rather than reading every operational database directly.

Candidate service	Responsibility	Independent operation
Learner	Profiles, status and identity validation.	Can manage learner records independently.
Course	Catalogue, course status and capacity definition.	Can publish course availability independently.
Registration	Uniqueness, capacity reservation and enrolment lifecycle.	Operates independently but queries cached learner/course eligibility.
Assessment	Assessment definitions, attempts and result strategies.	Can calculate and publish results independently.
Support	Ticket categorisation, priority and workflow.	Can continue accepting tickets during reporting outages.
Reporting	Bugzot events, metrics and management reports.	Consumes events asynchronously and tolerates temporary producer outages.

Communication: REST is appropriate for immediate validation and user-facing queries. Events such as registration-created, assessment-completed and ticket-updated reduce coupling for reporting and notification. Each message requires an idempotency key and correlation ID.

Testing and tracing: unit tests remain within each service; consumer-driven contract tests protect REST and event schemas; integration tests cover key workflows; and distributed traces connect gateway, service and event processing.
Bugzot becomes the central event and performance view while service-local logs remain available for diagnosis.

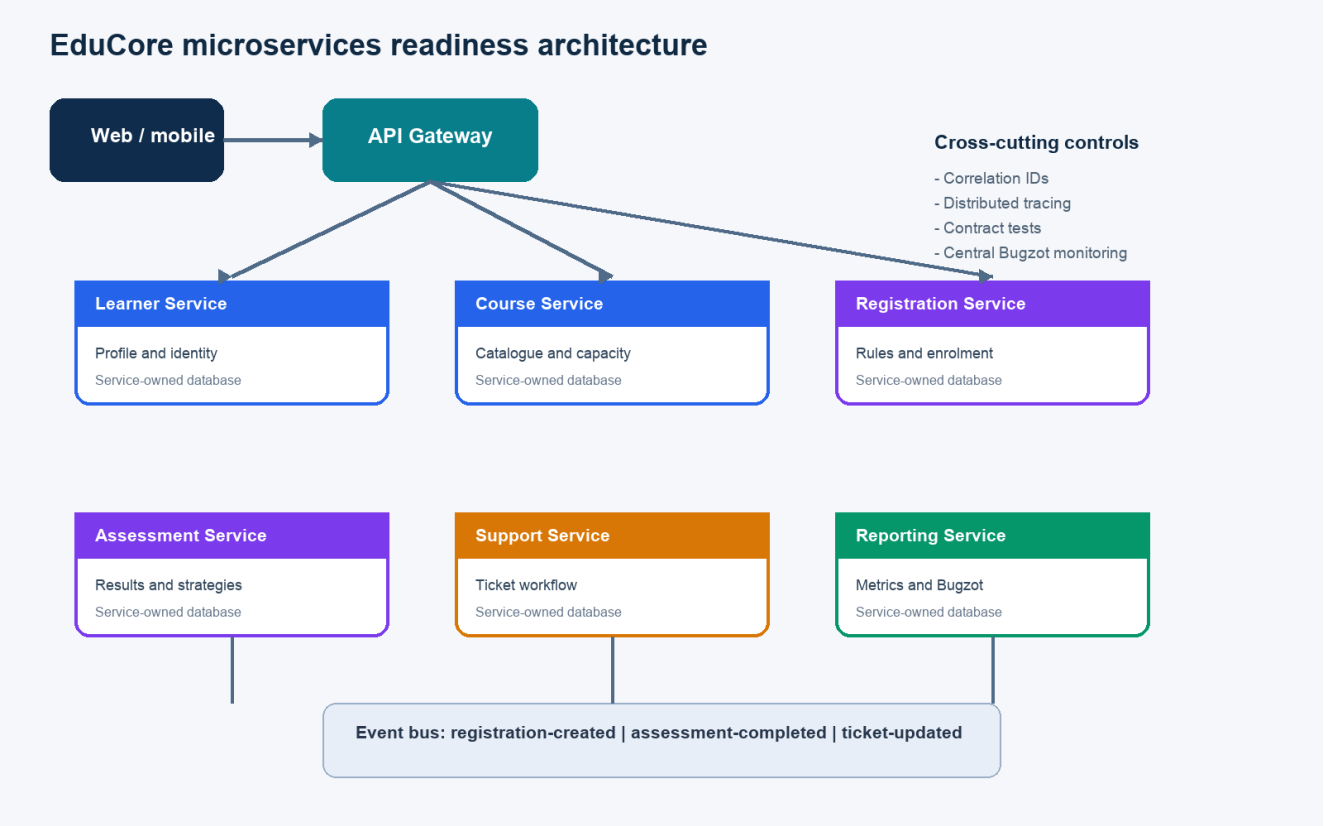

Figure 12. Proposed service boundaries, communication and cross-cutting controls.

Conclusion

EduCore demonstrates a maintainable enterprise-programming foundation rather than a collection of isolated examples. Validated entities and patterns support change; the registration service preserves uniqueness and capacity under contention; Bugzot provides diagnostic and performance visibility; tests and profiling supply reproducible quality evidence; and the proposed interfaces and service boundaries establish a credible path to future growth.

Submission verification

-Run python -m pytest --cov=educore and confirm all tests pass.

-Run python -m educore.demo and confirm the final registration count equals course capacity.

-Regenerate reports and compare the current evidence with the figures in this document.

-Review the AI-use disclosure for accuracy and retain the development history.

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Python Software Foundation (2026). concurrent.futures - Launching parallel tasks. Available at: https://docs.python.org/3/library/concurrent.futures.html

Python Software Foundation (2026). threading - Thread-based parallelism. Available at: https://docs.python.org/3/library/threading.html

Python Software Foundation (2026). The Python Profilers. Available at: https://docs.python.org/3/library/profile.html

pytest development team (2026). pytest documentation. Available at: https://docs.pytest.org/

Appendix A: AI-use statement

AI assistance was used for rubric analysis, project scaffolding, implementation support, test design, debugging, evidence preparation and report structuring. The accompanying AI_USE_LOG.md records the actual prompts and the student's required review and verification actions. AI use must be declared according to the final LMS template.

Appendix B: Reproduction commands

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install '.[dev]'
python -m pytest --cov=educore --cov-report=term-missing
python -m educore.demo --report-directory evidence/reports
```